From an earnings PDF to a saved report

Client → upload → validation → parsing → extraction → chunk preparation → persistence.

01–03 · CLIENT & SERVER

FLUTTER CLIENT — 01

Choose an earnings PDF

PDF bytes + ticker / period
Flutter file picker
Submit for ingestion

FASTAPI SERVER — 02

Receive the upload

POST /api/v1/ingestion/upload
multipart/form-data
Save PDF in data/uploads/

PYTHON + SQLITE — 03

Validate & fingerprint

Check %PDF- header + SHA-256
SQLite duplicate lookup
force=true skips cache reuse

PARSER

PYPDF · PYTHON — 04

Read page-aligned text

pypdf.PdfReader
Keep 1-based page numbers
Track blank pages

PYTHON CLASSIFIER — 05

Tag financial sections

Rules score headers + keywords
Cash flow, profitability, etc.
Retain page-domain metadata

EXTRACTOR

OPENAI — 06

Extract structured KPIs

gpt-4o-mini via LangChain
Selected financial pages
Pydantic schema + math rules

CHUNKER

LANGCHAIN — 07a

Recursive Chunker

1,000 chars · 100 overlap · non-blank pages

PYTHON — 07b

Metadata Enrichment

Ticker, period, page, domain + chunk ID

08 · PERSISTENCE

SQLITE · SQLALCHEMY — 08

Save report & KPIs

SQLite: stock_analyst.db
documents + financial_kpis
Persisted before indexing

CONTINUE ON SLIDE 02 → After saving, return the result and prepare the search indexes independently (proposed).

Return results while indexing continues

Proposed flow · after saving report + KPIs, fork into a response path and independent background indexing.

CONTINUED FROM SLIDE 01

SQLITE · SQLALCHEMY 08

Save report & KPIs

SQLite: stock_analyst.db
documents + financial_kpis
Persisted before indexing

RESPONSE NOW

Return without waiting for the search indexes.

FASTAPI RESPONSE 09

Return the result

Persisted report + KPIs
Response does not wait for indexing
Background work continues

FLUTTER + RIVERPOD 10

Refresh the dashboard

QuarterlyKpis.fromJson
Riverpod addAndSelectReport
Render scorecards + charts

BACKGROUND INDEXER

Independent execution · no join back to the response.

OPENAI + CHROMA 11

Embed & store in Chroma

text-embedding-3-small · 512-dim
LangChain Documents + metadata
Chroma: earnings_chunks

LANGCHAIN · BM25 12

Build the BM25 index

BM25Retriever.from_documents
Keyword ranking; no embeddings
Corpus: data/bm25/documents.json

PROPOSED FLOW · Current code still indexes before returning; independent background execution requires a server change.

What makes ingestion useful for RAG?

Keep small, traceable pieces of evidence that the vector and keyword indexes can search.

PARSER

PYPDF · PYTHON — 01

Keep page boundaries

pypdf extracts page text
Page numbers start at 1
Blank pages are skipped later

PYTHON CLASSIFIER — 02

Keep financial context

Rule-based domain labels
Headers + body keywords
Each page keeps its domain

EXTRACTOR

OPENAI · GPT-4O-MINI — 03

Resolve report identity

KPIs supply canonical metadata
Ticker + fiscal quarter + year
Statement pages sent to LLM

CHUNKER

LANGCHAIN — 04

Recursive Chunker

RecursiveCharacterTextSplitter
1,000 characters per chunk
100-character overlap

PYTHON ENRICHMENT — 05

Metadata Enrichment

Stable chunk ID + text
Ticker, period, page + domain
Chunk position + char count

Chunking uses all non-blank pages, including narrative pages outside the KPI extraction subset.

A SAVED CHUNK · actual inspection-artifact metadata

```
chunk_id: "TSLA_Q2_2026_p3_c0"
ticker / period: "TSLA" / "Q2" / 2026
page_number: 3
financial_domain: "cash_flow"
chunk position: 0 of 4 · char_count: 943
```

WHERE THE EVIDENCE LIVES TODAY

Chunks feed Chroma + BM25

03b_enriched_chunks_*.json / .md

Written when save_artifacts=true

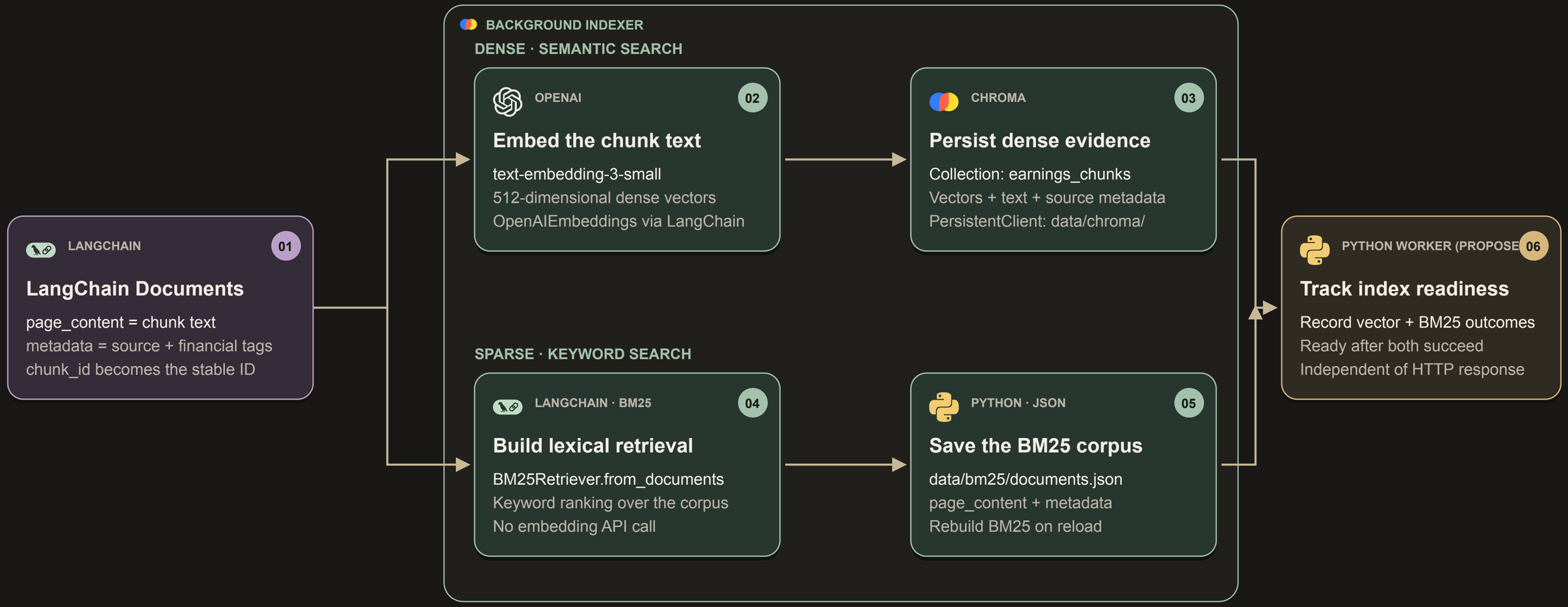
The Flutter upload enables artifact saving.

INDEXING HANDOFF · implemented in the ingestion orchestrator

Same enriched chunks → OpenAI embeddings + Chroma dense index, and LangChain BM25 sparse index

Background indexing after persistence

Proposed worker path · the saved report result is already returned while the indexes are being prepared.



Report availability and RAG search readiness are separate: the dashboard can load while indexing continues.

Vector and BM25 work may remain sequential inside the worker; neither blocks the response branch.

Proposed orchestration. Current code performs both indexing attempts inline before returning the result.